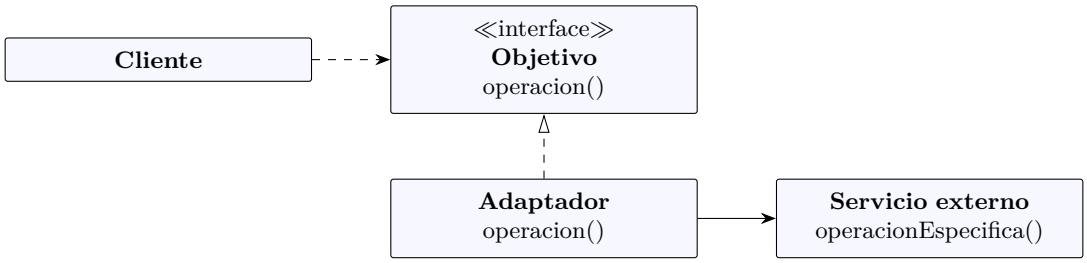
1. Adapter

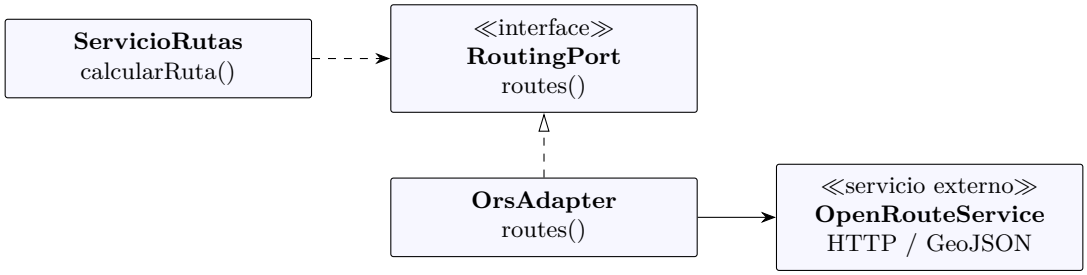
Problema en Vía. OpenRouteService emplea su propio formato de petición y GeoJSON de respuesta. El servicio de rutas necesita trabajar con lugares, geometría, distancia y duración en los tipos de la aplicación.

Aplicación prevista. `ServicioRutas` dependerá de `RoutingPort`. `OrsAdapter` implementará esa interfaz y traducirá la llamada a ORS y su respuesta. El spike ya ofrece esta implementación. La HU-14 la recibe como dependencia en sus pruebas. `OrsAdapter` también adapta geocodificación para otras historias.

UML genérico



UML aplicado a Vía



Ventajas. Aísla los formatos y errores de ORS, facilita probar el servicio y permite incorporar otro proveedor que cumpla la interfaz.

Inconvenientes. Exige mantener las conversiones y revisar qué conceptos son comunes a los proveedores. Un contrato demasiado específico de ORS dificultaría sustituirlo.

Alternativa. Llamar a ORS directamente desde el servicio requiere menos clases, pero mezcla reglas del proyecto con detalles HTTP y de GeoJSON. Adapter encaja por la dependencia externa que ya utiliza Vía.

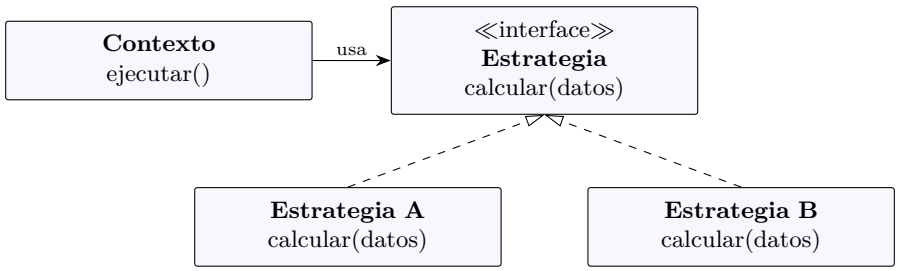
Evidencia. `src/adapters/providers.ts`, clase `OrsAdapter`; contrato en `src/domain/types.ts`. Los diagramas muestran el papel de rutas.

2. Strategy

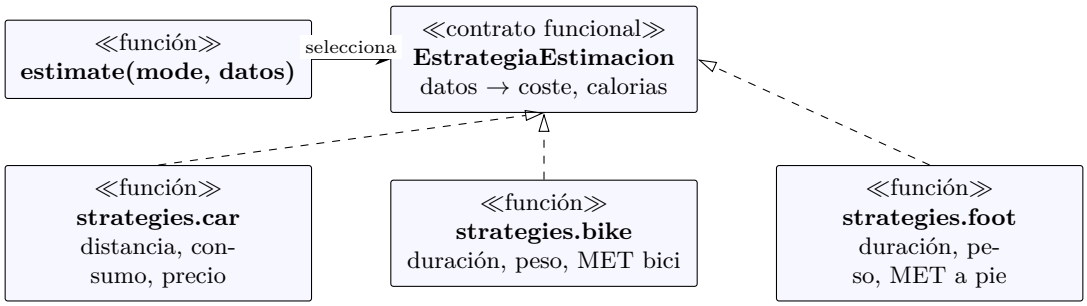
Problema en Vía. Los modos coche, bicicleta y a pie requieren reglas de estimación distintas. El coche combina distancia, consumo y precio energético. Bicicleta y marcha estiman calorías mediante duración, peso y el MET de cada actividad.

Aplicación prevista. La planificación seleccionará una estrategia según el transporte. Las HU-15, HU-16 y HU-17 usarán estos cálculos. En el spike, las estrategias son funciones del objeto `strategies` y `estimate` selecciona una de ellas. En TypeScript se puede expresar este patrón con funciones que compartan la misma firma.

UML genérico



UML aplicado a Vía



Ventajas. Localiza las reglas de cada transporte y permite comprobar los cálculos por separado. La planificación utiliza una operación común.

Inconvenientes. Hay que mantener un contrato coherente y decidir qué datos recibe cada estrategia. Añadir estrategias para diferencias mínimas puede complicar el diseño.

Alternativa. Un bloque condicional dentro del planificador sería suficiente con pocas reglas estables, pero concentraría allí cada nueva fórmula. Combustible y electricidad comparten la fórmula distancia/100 × consumo × precio; por eso se propone seleccionar por modo de transporte, como en el spike.

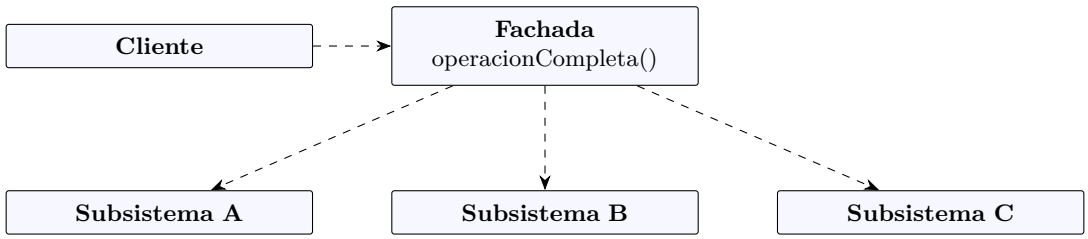
Evidencia. `src/domain/cost.ts`. `EstrategiaEstimacion` es el nombre del contrato común representado en el UML; las funciones del spike lo cumplen de forma estructural.

3. Facade

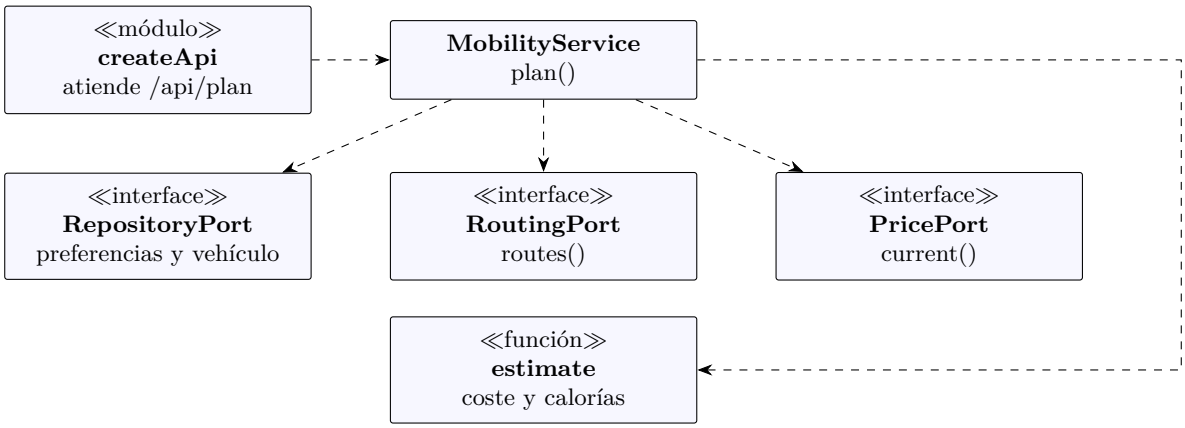
Problema en Vía. Planificar un viaje implica comprobar los extremos, recuperar preferencias y vehículo, consultar precios y rutas, calcular estimaciones y ordenar alternativas. Repetir esa coordinación en cada consumidor aumentaría el acoplamiento.

Aplicación prevista. Una operación de planificación ofrecerá al cliente un resultado conjunto. La HU-14 aporta las rutas; las HU-15 a HU-18 incorporan estimaciones y criterio de selección. En el spike, `MobilityService.plan` ya realiza esa coordinación.

UML genérico



UML aplicado a Vía



Ventajas. Centraliza la secuencia de planificación y ofrece un punto de entrada sencillo a los consumidores. Los componentes especializados conservan sus cálculos y consultas.

Inconvenientes. La fachada puede crecer demasiado si absorbe todas las responsabilidades. El spike combina muchas operaciones en `MobilityService`; en el proyecto se vigilará su tamaño y se delegará el trabajo especializado.

Alternativa. Coordinar desde el manejador HTTP simplifica inicialmente la estructura, pero mezcla protocolo y planificación. Una fachada permite reutilizar la operación desde otros consumidores.

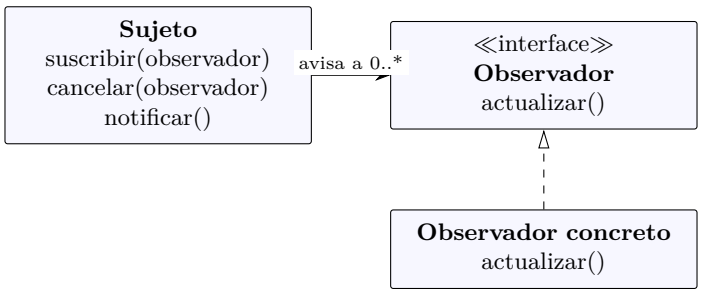
Evidencia. `src/services.ts`, método `plan`; entrada HTTP en `src/http.ts`. Se muestra la parte de planificación del servicio.

4. Observer

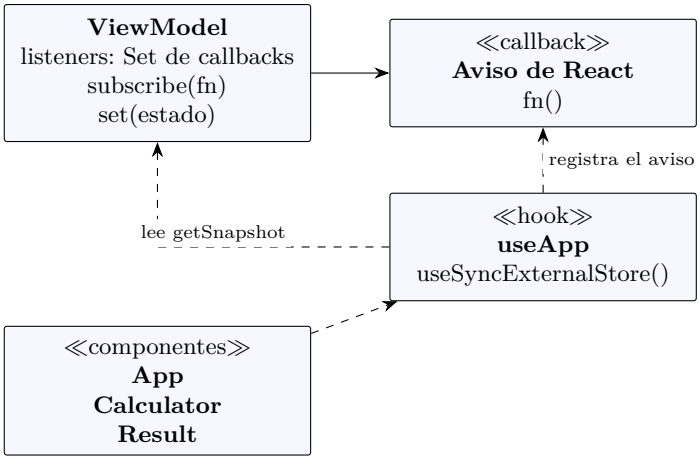
Problema en Vía. Varias vistas consumen el estado compartido del usuario y de la planificación. Al modificarlo, las vistas activas necesitan enterarse para mostrar los datos actuales. El modelo necesita permitir la suscripción y su cancelación.

Aplicación prevista. Se conservará un modelo observable para las vistas. En el spike, `ViewModel.subscribe` guarda callbacks y devuelve una función para cancelar la suscripción. `set` sustituye el estado y ejecuta los callbacks. El hook `useApp` lo enlaza con React.

UML genérico



UML aplicado a Vía



Ventajas. Las vistas se actualizan tras cambios como guardar lugares o recibir rutas. El modelo mantiene callbacks en lugar de referencias a componentes concretos.

Inconvenientes. Las suscripciones deben cancelarse al desmontar las vistas. Un estado muy amplio puede provocar actualizaciones de más componentes de los necesarios.

Alternativa. Mantener todo el estado en un componente padre y pasarlo por propiedades puede servir con una interfaz pequeña. El spike ya comparte un modelo y tres componentes se suscriben mediante el hook.

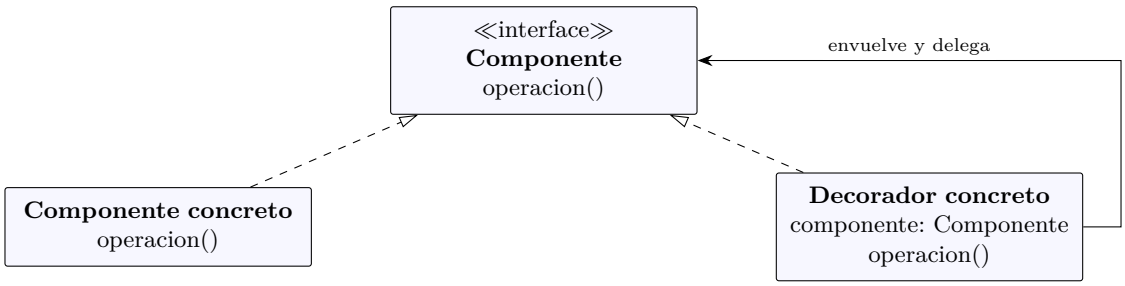
Evidencia. `web/model.ts`, `ViewModel` y `useApp`; consumidores en `web/main.tsx`. Los nodos marcados como hook, callback y componentes representan funciones de TypeScript.

5. Decorator

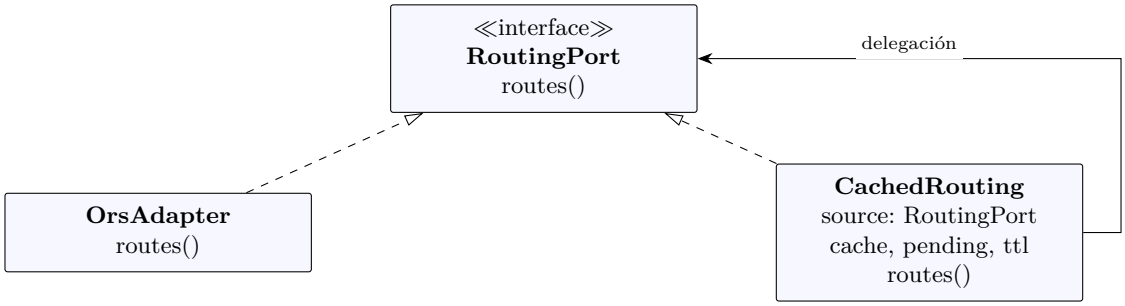
Problema en Vía. Repetir una consulta de ruta con los mismos extremos y modo implica otra llamada externa y espera. Varias solicitudes simultáneas iguales también pueden duplicar la consulta. Se quiere añadir una política de caché al proveedor elegido.

Aplicación prevista. `CachedRouting` envolverá un `RoutingPort` y ofrecerá el mismo método `routes`. En el spike guarda hasta cien resultados recientes durante tres minutos, agrupa solicitudes simultáneas iguales y elimina las entradas caducadas al consultarlas. Es una decisión técnica de optimización que se propone conservar.

UML genérico



UML aplicado a Vía



Colaboración. El servidor crea `new CachedRouting(ors)`. La clave incluye coordenadas de origen y destino y el modo. Si el resultado sigue vigente se devuelve una copia. En otro caso se delega en el proveedor. Los fallos no se conservan en caché.

Ventajas. Reduce consultas repetidas y tiempo de espera manteniendo el contrato del proveedor. La política se configura al envolverlo.

Inconvenientes. Consume memoria y puede devolver un resultado anterior hasta que caduque. La caducidad debe adecuarse al tipo de información consultada.

Alternativa. Incorporar la caché dentro de `OrsAdapter` reduciría el número de clases, pero uniría política de caché y adaptación de ORS. Decorator permite reutilizar la política con otro `RoutingPort`.

Evidencia. `src/adapters/providers.ts`, `CachedRouting`; composición en `src/server.ts`. Existen comprobaciones de caché en `tests/adapters.test.ts`.

Alcance del documento. Análisis preliminar para la Entrega 2, basado en las 26 HU y en el spike. El SUT nuevo de la IT1 sigue en fase roja. La justificación del patrón de interfaz se preparará en su apartado propio.